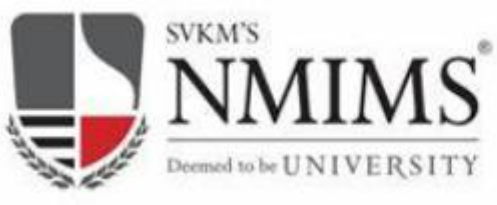
SVKM's NMIMS University
School of Technology Management & Engineering
COURSE: Mobile Application Development



Mumbai

LAB Manual

Mobile Application Development

Subject Code - 702AI0E002

Submitted by: Jivitesh Dutt | Roll No: N026 | MBATech CE (Batch D1)

School of Technology Management & Engineering
SVKM'S-NMIMS, Mumbai
[2024-25]

SVKM's NMIMS University
School of Technology Management & Engineering
COURSE: Mobile Application Development

LAB ASSIGNMENT-1

Instruction: After checkout, create a branch with a name starting with your roll number. Complete the assignment, push your changes, and create a pull request. The best implementation will be merged with my repository.

To build upon the Gemini Jetpack Compose starter app and enhance its UI and functionality, follow this step-by-step guide. Starter repository:

github.com/ifahimkhan/GeminiApiComposeStarter

1. **Obtain a Gemini API Key**
   - **Access Google AI Studio**: Visit Google AI Studio to request a Gemini API key.
   - **Generate the API Key**: Follow the prompts to create your API key.
   - **Store the Key Locally**: Open local.properties in the project root (it is git-ignored) and add the line below. Gradle reads this file at build time and exposes the value as BuildConfig.GEMINI_API_KEY. Never paste the key into Kotlin files, strings.xml or build.gradle.kts.

     ```
     GEMINI_API_KEY=your_api_key_here
     ```

ai-studio

2. **Explore and Understand the Starter App**
   - **Review the Codebase**: Familiarise yourself with the Compose structure: MainActivity sets the content, ui/chat/ChatScreen.kt holds the composables, ChatViewModel.kt and ChatUiState.kt hold the UI state, and data/GeminiRepository.kt talks to the Gemini API.

- **Run the App**: Use the Android Emulator to run the app and observe its current functionality.

3. **Secure the API Key (Mandatory)**

- **Never Hardcode**: The key must not appear as a string literal in any Kotlin file, XML resource or committed Gradle file. Reviewers will search your pull request for it.
- **Keep It Out of Git**: local.properties is already listed in .gitignore; do not remove that entry. Before every commit run git status and confirm local.properties is not staged. Add a local.properties.example file with a placeholder value so teammates know what to configure. (Optional) add a pre-commit secret scanner such as gitleaks.
- **Environment Fallback for CI**: In app/build.gradle.kts, fall back to an environment variable when local.properties has no key, so CI builds use a repository secret instead of a file:

```
val geminiApiKey = localProperties.getProperty("GEMINI_API_KEY")
    ?: System.getenv("GEMINI_API_KEY") ?: ""
```

- **Encrypt at Rest**: On first launch, generate an AES-256-GCM key inside the Android Keystore (KeyGenParameterSpec) and use it to encrypt the API key. Persist only the ciphertext in DataStore or EncryptedSharedPreferences. Decrypt in memory only at the moment the GenerativeModel is created, and never log, toast or display the decrypted value.
- **Obfuscate the Release Build**: Set isMinifyEnabled = true so R8 obfuscates the release APK and the key is not trivially readable from decompiled code.
- **Know the Limits**: Client-side encryption raises the bar but cannot fully hide a key from a determined attacker. In your README, explain how a production app would move Gemini calls behind a backend proxy or protect them with Firebase App Check and restricted API keys.

4. **Implement UI Enhancements (Jetpack Compose)**
    - **Use LazyColumn**: Render the conversation as a LazyColumn of Material 3 chat bubbles (user vs. Gemini) with stable keys, and auto-scroll to the latest message.
    - **State Hoisting**: Keep all UI state in ChatUiState exposed as a StateFlow from ChatViewModel, collect it with collectAsStateWithLifecycle(), and keep composables stateless.
    - **Responsive Layouts**: Use WindowSizeClass and adaptive modifiers so the layout adapts to phones, tablets and orientation changes.
    - **Loading and Error States**: Show a CircularProgressIndicator while waiting for a response and a Snackbar or error card when the API call fails.
    - **Dark Mode Support**: Use isSystemInDarkTheme() and Material 3 dynamic colour in Theme.kt to support low-light environments. (optional)
5. **Enhance Functionality**
    - **Expand Input Modalities**: Allow users to input queries via voice (speech to text) using RecognizerIntent launched with rememberLauncherForActivityResult.
    - **Personalized User Experience**: Remember user preferences with Preferences DataStore and past conversations with Room so that chat history survives app restarts and provides a more tailored experience.
6. **Testing and Optimization**
    - **Conduct Thorough Testing**: Unit-test ChatViewModel with kotlinx-coroutines-test and a fake GeminiRepository; write Compose UI tests with createComposeRule() for the chat screen.

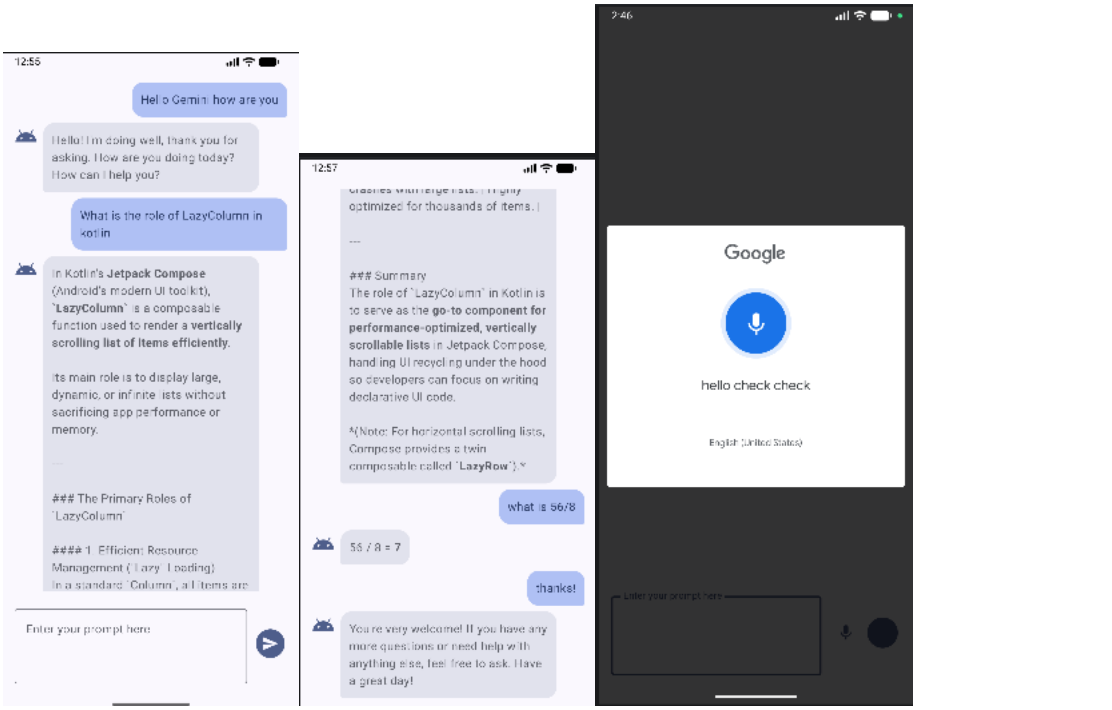
- **Optimize Performance**: Use Layout Inspector recomposition counts to remove unnecessary recompositions, keep network calls off the main thread, and focus on reducing latency and smooth user interactions.

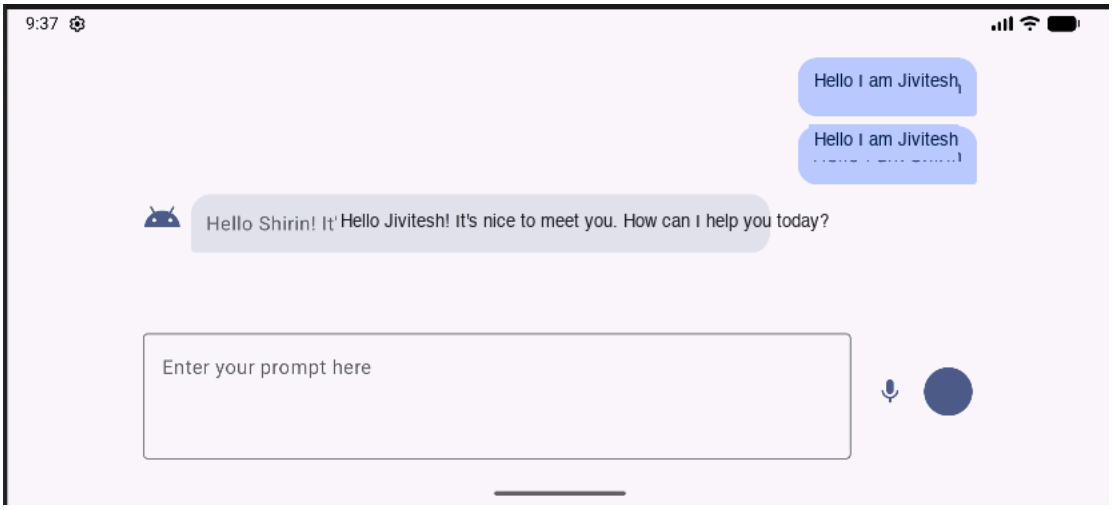
7. **Submission Checklist**

- **Branch and Pull Request**: Branch name starts with your roll number; the PR description lists implemented features with screenshots.
- **No Secrets in History**: The PR diff and git log contain no API key and no local.properties file.
- **README**: Explains where to place the key, how the encryption flow works, and how to run the tests.

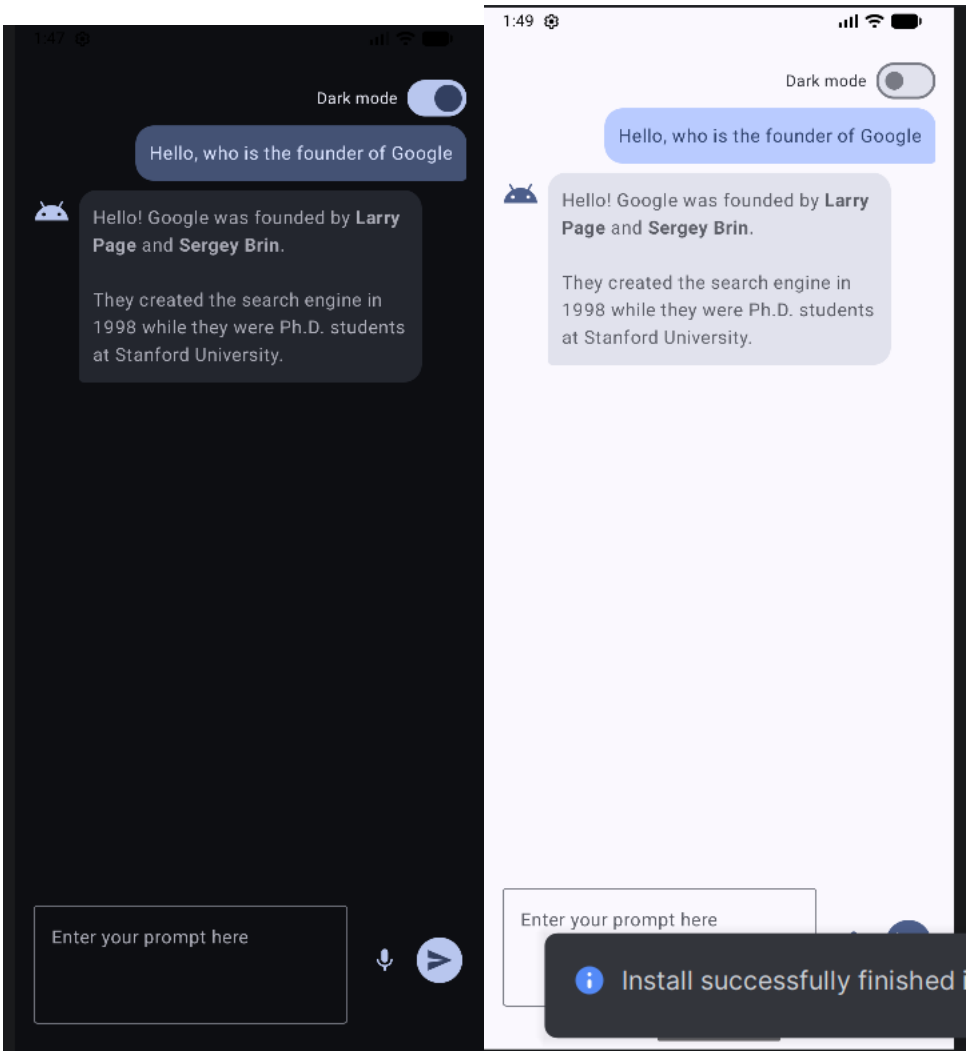
Repo link: N026_JiviteshDutt-Gemini Chat App-Assignment1 by jiviteshhhh · Pull Request #158 · ifahimkhan/GeminiApiComposeStarter

SVKM's NMIMS University
School of Technology Management & Engineering
COURSE: Mobile Application Development





SVKM's NMIMS University
School of Technology Management & Engineering
COURSE: Mobile Application Development



SVKM's NMIMS University
School of Technology Management & Engineering
COURSE: Mobile Application Development

